

Chatbot com interface gráfica em Python com áudio gravação transcrição e resposta por voz

```
import tkinter as tk # Importa a biblioteca Tkinter para criar interfaces gráficas
from tkinter import PhotoImage # Importa suporte para imagens no Tkinter
import keyboard # Permite capturar teclas do teclado (atalhos globais)
from PIL import Image, ImageTk # Biblioteca para manipulação de imagens (Pillow)
import pyaudio # Biblioteca para gravação e reprodução de áudio
import threading # Permite executar tarefas em paralelo (threads)
from openai import OpenAI # Cliente da API da OpenAI
import wave # Manipulação de ficheiros de áudio WAV
import io # Manipulação de streams de dados em memória
import json # Manipulação de dados em formato JSON
import os # Interação com o sistema operativo
from dotenv import load_dotenv # Carrega variáveis de ambiente de um ficheiro
.env

import wave # (Repetido) biblioteca para ficheiros WAV
import pyaudio # (Repetido) biblioteca de áudio
import mss # Captura de ecrã (screenshots)
import base64 # Codificação de dados em base64
from io import BytesIO # Permite trabalhar com dados binários em memória
import re # Expressões regulares
import matplotlib.pyplot as plt # Biblioteca para gráficos
import pyautogui # Automação de rato e teclado
import replicate # API para correr modelos de IA externos
```

```
# Carrega variáveis de ambiente do ficheiro .env
```

```
load_dotenv()
```

```
# Configurações de gravação de áudio
```

```
CHUNK = 1024 # Número de frames por buffer
```

```
FORMAT = pyaudio.paInt16 # Formato de áudio (16 bits)
```

```
CHANNELS = 1 # Número de canais (mono)
```

```
RATE = 16000 # Taxa de amostragem (Hz)
```

```
client = OpenAI() # Cria cliente da OpenAI
```

```
class ChatbotApp: # Classe principal da aplicação
```

```
    def __init__(self, root):
```

```
        self.root = root # Janela principal
```

```
        self.root.title("MYPC") # Título da janela
```

```
        self.root.configure(bg='white') # Cor de fundo
```

```
        self.settings_file = "settings.json" # Ficheiro de configurações
```

```
        self.settings = self.load_settings() # Carrega configurações guardadas
```

```
        # Checkbox para manter a janela sempre no topo
```

```
        self.always_on_top_var = tk.BooleanVar(value=self.settings.get("always_on_top",  
True))
```

```
self.always_on_top_checkbox = tk.Checkbutton(root, text="Always on Top",
variable=self.always_on_top_var, command=self.toggle_always_on_top)

self.always_on_top_checkbox.pack(pady=5)

self.root.attributes('-topmost', self.always_on_top_var.get()) # Define se fica
sempre no topo


# Carrega imagem
script_dir = os.path.dirname(__file__) # Diretório do script
image_path = os.path.join(script_dir, "junin.jfif") # Caminho da imagem
image = Image.open(image_path) # Abre imagem
image.thumbnail((400, 400), Image.LANCZOS) # Redimensiona
self.root.geometry(f'{image.width + 50}x{image.height + 450}+100+100') #
Define tamanho da janela

self.photo = ImageTk.PhotoImage(image) # Converte para Tkinter
self.label = tk.Label(root, image=self.photo, bg='white') # Label com imagem
self.label.pack(side="top", pady=10)


# Área de chat

self.chat_display = tk.Text(root, height=10, width=50, state='disabled',
wrap='word', bg='lightgrey')
self.chat_display.pack(pady=10)


# Campo de input do utilizador
self.user_input = tk.Entry(root, width=50)
self.user_input.pack(pady=5)
```

```
self.user_input.bind("<Return>", self.send_message) # Enviar ao pressionar
Enter

# Botão de gravação

self.record_button = tk.Button(root, text="Click to Record",
command=self.toggle_recording, bg='blue', fg='white')

self.record_button.pack(pady=5)

keyboard.add_hotkey('ctrl+alt', self.toggle_recording) # Atalho global


# Checkbox para ouvir resposta

self.hear_response_var = tk.BooleanVar(value=self.settings.get("hear_response",
False))

self.hear_response_checkbox = tk.Checkbutton(root, text="Hear the response",
variable=self.hear_response_var)

self.hear_response_checkbox.pack(pady=5)


# Dropdown de vozes

self.voice_var = tk.StringVar(value=self.settings.get("selected_voice", "alloy"))
self.voice_label = tk.Label(root, text="Select Voice:")
self.voice_label.pack(pady=5)

self.voice_dropdown = tk.OptionMenu(root, self.voice_var, "alloy", "echo",
"fable", "onyx", "nova", "shimmer")

self.voice_dropdown.pack(pady=5, padx=10)


# Inicialização do PyAudio

self.p = pyaudio.PyAudio()
```

```
self.audio_stream = None

self.is_recording = False

self.chat_history = [] # Histórico de mensagens


self.root.protocol("WM_DELETE_WINDOW", self.on_closing) # Evento ao fechar
janela


def load_settings(self):
    if os.path.exists(self.settings_file): # Se o ficheiro existe
        with open(self.settings_file, 'r') as file:
            return json.load(file) # Carrega JSON
    return {} # Caso contrário retorna vazio


def save_settings(self):
    self.settings["selected_voice"] = self.voice_var.get() # Guarda voz
    self.settings["always_on_top"] = self.always_on_top_var.get() # Guarda estado
    topo
    self.settings["hear_response"] = self.hear_response_var.get() # Guarda opção
    de áudio
    with open(self.settings_file, 'w') as file:
        json.dump(self.settings, file) # Escreve no ficheiro


def toggle_always_on_top(self):
    self.root.attributes('-topmost', self.always_on_top_var.get()) # Alterna topo
```

```
def toggle_recording(self, event=None):
    if self.is_recording: # Se já está a gravar
        self.stop_recording()
        self.record_button.config(text="Click to Record", bg='blue')
    else:
        self.is_recording = True
        self.record_button.config(text="Recording...", bg='red')
        self.start_recording()

def start_recording(self):
    self.frames = [] # Lista de frames de áudio
    self.audio_stream = self.p.open(format=FORMAT, channels=CHANNELS,
rate=RATE, input=True, frames_per_buffer=CHUNK)

    self.recording_thread = threading.Thread(target=self.record) # Thread de
gravação
    self.recording_thread.start()

def record(self):
    while self.is_recording:
        data = self.audio_stream.read(CHUNK) # Lê áudio
        self.frames.append(data)

    self.audio_stream.stop_stream()
    self.audio_stream.close()
```

```
audio_data = b''.join(self.frames) # Junta frames
audio_file = io.BytesIO()
```

```
with wave.open(audio_file, 'wb') as wf:
    wf.setnchannels(CHANNELS)
    wf.setsampwidth(self.p.get_sample_size(FORMAT))
    wf.setframerate(RATE)
    wf.writeframes(audio_data)
```

```
audio_file.name = "output.wav"
```

```
transcript = self.transcribe_audio(audio_file) # Transcrição
if transcript:
    self.display_message("You: " + transcript)
    response = self.get_response(transcript)
    self.display_message("Bot: " + response)
    if self.hear_response_var.get():
        self.speak_response(response)
```

```
def stop_recording(self):
    self.is_recording = False # Para gravação
```

```
def transcribe_audio(self, audio_file):
    try:
        audio_file.seek(0)
```

```
response = client.audio.transcriptions.create(model="whisper-1",  
file=audio_file)
```

```
return response.text
```

```
except Exception as e:
```

```
    print("Erro:", e)
```

```
    return ""
```

```
def get_chat_history(self):
```

```
    return "\n".join(self.chat_history) # Junta histórico
```

```
def display_message(self, message):
```

```
    self.chat_display.config(state='normal')
```

```
    self.chat_display.insert(tk.END, message + "\n") # Mostra mensagem
```

```
    self.chat_history.append(message)
```

```
    self.chat_display.config(state='disabled')
```

```
    self.chat_display.see(tk.END)
```

```
def send_message(self, event=None):
```

```
    user_message = self.user_input.get().strip()
```

```
    if user_message:
```

```
        self.display_message("You: " + user_message)
```

```
        self.user_input.delete(0, tk.END)
```

```
        response = self.get_response(user_message)
```

```
        self.display_message("Bot: " + response)
```

```
        if self.hear_response_var.get():
```

```
self.speak_response(response)
```

```
def on_closing(self):
```

```
    keyboard.unhook_all_hotkeys() # Remove atalhos
```

```
    self.save_settings() # Guarda definições
```

```
    self.root.destroy() # Fecha janela
```

```
# Execução principal
```

```
if __name__ == "__main__":
```

```
    root = tk.Tk() # Cria janela
```

```
    app = ChatbotApp(root) # Instancia app
```

```
    root.mainloop() # Inicia loop da interface
```